

CASE STUDY 3

Tracking an Active ClickFix / EtherHiding Operation

Reverse Engineering BW Panel Stateful Delivery Across Compromised Italian Web Infrastructure

Author: Andrea Salvadori

Research alias: BresshSec

Research type: Independent Threat Research

Research freeze: 14 September 2026

Classification: Public Threat Research / Defensive Analysis

Abstract

This research documents the investigation of an active web-based malware delivery operation observed through compromised Italian web infrastructure and exhibiting characteristics associated with the ClickFix, EtherHiding and BW Panel / ErrTraffic ecosystem (the term “BW Panel” is used throughout this report as a working label and is not a name confirmed by the operators or by prior public research). The protocol-level evidence developed in this paper is centered on one deep-dive sandbox sample.

The investigation began with the analysis of compromised websites displaying suspicious client-side behavior and progressively expanded into the examination of obfuscated JavaScript loaders, Polygon blockchain JSON-RPC communications, smart-contract-based infrastructure resolution, rotating backend infrastructure and the client protocol used by the downstream delivery framework.

The analyzed infection chain demonstrated a separation between the compromised website, the blockchain-based control layer and the downstream delivery infrastructure.

A compromised page executed an obfuscated JavaScript loader which performed an eth_call against the Polygon network. The call used contract:

0xB6bC9e1D0b2fB96Ab7C47E04Cb0BE477410bC1f2

and function selector:

0xb68d1809

The returned ABI-encoded data resolved downstream infrastructure used by the delivery framework.

During the primary observation, this mechanism ultimately led to:

fingerprint-veri[.]info

where additional JavaScript modules and client communication were observed.

The investigation subsequently identified a protocol-level behavior that became the principal focus of the research.

A shared value:

8402fb1338daab6a166e91aa8c92a20798acfa98fb75c5a2

was observed first as a module retrieval parameter:

/api.php?s=<K>

and later reused inside subsequent client communication as:

k=<K>

The client also transmitted a Base64-encoded structure containing separate fields representing operating-system information, victim-site context, a UUID-shaped client identifier and a separate ray value.

Most notably, during the same sandbox session, requests using the same module key initially returned substantial JavaScript modules of approximately 91 KB and 103 KB, while subsequent requests returned exactly 17 bytes of syntactically valid but intentionally inert JavaScript:

```
(function(){})();
```

This demonstrates that, within the observed session, module delivery was not a static mapping between a key and a payload.

Instead, the backend altered its response according to state not directly visible to the analyst.

The exact server-side condition responsible for this transition remains unknown.

This report therefore documents an under-described stateful behavior of an active BW Panel client, while deliberately avoiding claims unsupported by the available evidence regarding initial access, a specific exploited vulnerability or attribution to a named threat actor.

Research at a Glance

Primary finding: during the same sandbox session, the same module key transitioned from substantial active JavaScript delivery to the explicit 17-byte no-op `(function(){})();``. The evidence supports state-dependent delivery in the observed session, while the exact server-side gating condition remains unknown.

Scope discipline: the paper does not claim discovery of ClickFix, EtherHiding, BW Panel, a universal BW Panel state model, a common exploited CVE, or attribution to a named APT.

1. Introduction

ClickFix has become an increasingly important social-engineering technique in modern malware delivery.

Rather than relying exclusively on automatic exploitation, ClickFix campaigns convince users to execute attacker-provided commands themselves.

A typical interaction presents the victim with a fake CAPTCHA, browser verification prompt or technical error message.

The victim is instructed to perform actions such as:

Win + R

followed by:

Ctrl + V

and finally:

Enter

The command placed into the clipboard by the malicious page can then launch native Windows utilities such as PowerShell, mshta.exe, cmd.exe or other trusted binaries.

This technique is particularly effective because part of the execution chain is initiated manually by the victim.

Throughout this report, “BW Panel” is used as a working label for the observed backend/delivery framework accessed via /api.php; it is not a name confirmed by the operators or by prior public research. The present investigation did not begin as an attempt to study BW Panel itself.

Instead, the research originated from the analysis of compromised Italian web infrastructure associated with ClickFix behavior.

As additional samples were collected and compared, common structural characteristics began to emerge.

These included:

- heavily obfuscated JavaScript;
- dynamic JavaScript execution;
- blockchain RPC communication;
- Polygon smart-contract interaction;
- dynamically resolved downstream infrastructure;
- long hexadecimal configuration values;

- additional JavaScript modules retrieved from /api.php;
- ClickFix presentation logic.

The investigation therefore moved from conventional IOC analysis toward protocol and framework analysis.

That transition is important.

Domains and IP addresses used by malware delivery infrastructure can change quickly.

Protocol structure, client logic and state-management mechanisms tend to be more durable.

The primary objective consequently became:

Identify which characteristics remain stable when infrastructure and superficial JavaScript implementation details change.

2. Research Objectives

The investigation attempted to answer several questions.

2.1 Loader architecture

Do apparently different compromised websites execute unrelated malware, or do their loaders share structural characteristics?

2.2 Infrastructure resolution

How does the injected JavaScript discover the infrastructure from which subsequent modules are retrieved?

2.3 Blockchain usage

What role does Polygon play in the delivery chain?

Is blockchain infrastructure merely telemetry, or does it actively participate in resolving attacker infrastructure?

2.4 Delivery keys

What do the long hexadecimal values passed to the panel represent?

Are they victim identifiers, campaign identifiers, module identifiers or configuration keys?

2.5 Client state

Does the delivery framework maintain state between requests?

2.6 Cross-victim behavior

Are identical protocol characteristics observable across unrelated compromised websites?

2.7 Initial access

Can the compromise of the Italian websites be linked to a common WordPress vulnerability or infrastructure weakness?

2.8 Attribution

Does the available evidence support attribution to a specific threat actor or APT?

These questions were treated separately because infrastructure attribution, malware-family attribution and initial-access attribution require different evidence.

3. Research Methodology

The investigation used a defensive and non-intrusive methodology.

Analysis was restricted to:

- public threat-intelligence sources;
- public sandbox executions;
- captured HTTP traffic;
- JavaScript retrieved during sandbox analysis;
- static deobfuscation;
- read-only blockchain information;
- public infrastructure information;
- comparison with existing security research.

No attempt was made to gain unauthorized access to compromised websites or attacker-controlled infrastructure.

No credentials were tested.

No exploitation was performed.

No persistence mechanisms were deployed.

The analysis concentrated on artifacts already exposed to browsers or captured inside isolated sandbox environments.

The methodology followed a repeatable sequence:

Capture → Normalize → Decode → Compare → Correlate → Validate → Classify confidence

This sequence was intentionally designed to prevent assumptions made during early analysis from becoming conclusions later in the research.

4. Confidence Model

To prevent analytical conclusions from becoming stronger than the evidence supporting them, findings were divided into four categories.

CONFIRMED

Directly observed in captured traffic, source code, sandbox execution or decoded data.

CORROBORATED

Observed directly and independently supported by external research or additional public samples.

ASSESSED

A technically supported interpretation for which direct proof is incomplete.

UNKNOWN

The available evidence is insufficient to reach a reliable conclusion.

A fifth category is used where appropriate:

NOT SUPPORTED

A hypothesis was considered but available evidence does not justify maintaining it as a conclusion.

This distinction is particularly important for initial-access and threat-actor attribution.

4.1 Scope of Evidence in This Paper

This paper is intentionally evidence-centered on the danielennittisotres[.]it sandbox sample for the protocol-level findings developed in Sections 5-24. Related earlier investigations informed the research direction, but their technical evidence is not imported into this document unless explicitly reproduced and cited here. Accordingly, this paper does not assign confidence labels, compromise status, or protocol findings to those external samples.

5. Primary Deep-Dive Investigation

The primary technical evidence developed in this paper comes from a deep-dive sandbox analysis of:

danielennittisotres[.]it

The ANY.RUN report was generated on 14 September 2026 using a Windows 10 Professional virtual machine.

ANY.RUN classified the activity as:

Malicious activity

and applied the tags:

phishing

clickfix

terminalfix

etherhiding

obfuscated-js

The sandbox recorded 101 URLs during execution.

Among the most relevant were:

fingerprint-veri[.]info

multiple Polygon RPC providers, including:

polygon.gateway.tenderly.co

1rpc.io/matic

rpc-mainnet.matic.quiknode.pro

and:

polygon-public.nodies.app

These appear directly in the captured URL list.

The combination was immediately notable because normal browsing of the analyzed website did not provide an obvious functional reason for browser-side interaction with multiple Polygon RPC providers.

6. First Delivery Requests

Two particularly important requests appeared in the sandbox traffic.

The first contained:

`/api.php?s=6c720afc26340f39e9f29e7b7f48d8581b0b5f3c1f4d4716`

and the second:

`/api.php?s=8402fb1338daab6a166e91aa8c92a20798acfa98fb75c5a2`

Both are recorded directly by the sandbox report.

This became an important clue.

If `s=` represented simply a unique identifier for the compromised website, observing multiple values during one execution would require additional explanation.

Instead, later analysis showed that at least one of these values was embedded inside the retrieved JavaScript itself and reused across different protocol stages.

For clarity, the primary value will subsequently be referred to as:

K

where:

K = 8402fb1338daab6a166e91aa8c92a20798acfa98fb75c5a2

7. JavaScript Obfuscation

The retrieved JavaScript was intentionally obfuscated.

One representative wrapper followed approximately this structure:

1. define an integer array;
2. XOR each integer with a constant;
3. reconstruct a string;
4. Base64-decode the result using `atob()`;
5. XOR the decoded characters using a second constant;
6. reconstruct the actual JavaScript;
7. execute it dynamically using `new Function()`.

Conceptually:

integer array

|

v

XOR layer #1

|

v

Base64 encoded string

|

v

atob()

|

v

XOR layer #2

|

v

decoded JavaScript

|

v

new Function(code)

Different captured loaders used different variable names, constants and numerical arrays.

This initially made them appear unrelated.

However, comparison of the decoded behavior showed that superficial polymorphism concealed a more stable architecture.

8. Polymorphism Versus Structural Invariants

Several elements changed between samples:

- **variable names;**
- **XOR constants;**
- **guard values;**
- **array contents;**
- **delivery keys;**

- ordering of RPC endpoints.

Other characteristics remained significantly more stable:

- JavaScript-based decoding;
- dynamic execution;
- multiple Polygon RPC providers;
- JSON-RPC;
- eth_call;
- ABI decoding;
- the same smart-contract interaction pattern;
- selector 0xb68d1809.

This distinction is important for defensive detection.

A signature built around one XOR key would be fragile.

A detection correlating:

browser JavaScript

+

multiple Polygon RPC endpoints

+

eth_call

+

0xb68d1809

+

ABI URL decoding

+

dynamic JavaScript retrieval

is considerably more resilient.

Figure 1 - Evidence-backed EtherHiding resolution chain

danielennittisotres[.]it (deep-dive sandbox sample)

|

v

```

Obfuscated JavaScript loader
|
v
Polygon JSON-RPC -> eth_call
|
+-- Contract: 0xB6bC9e1D0b2fB96Ab7C47E04Cb0BE477410bC1f2
+-- Selector: 0xb68d1809
|
v
ABI-encoded response
|
v
fingerprint-veri[.]info
|
v
/api.php?s=K -> BW Panel client

```

Figure 1. Reconstruction limited to evidence used in this paper; no operator-wallet or unverified update-count claims are included.

9. EtherHiding

The loader communicated with the Polygon blockchain using JSON-RPC.

The relevant method was:

eth_call

The observed request targeted:

0xB6bC9e1D0b2fB96Ab7C47E04Cb0BE477410bC1f2

with:

data = 0xb68d1809

This is the corrected canonical contract address used throughout this report: 40 hexadecimal characters after 0x.

The response was ABI-encoded.

One captured result was:

0x0020
001d
68747470733a2f2f66696e6765727072696e742d766572692e696e6666f000000

The final hexadecimal sequence decodes to:

<https://fingerprint-veri.info>

This established an important relationship:

Compromised website

|

v

Injected loader

|

v

Polygon JSON-RPC

|

v

eth_call

|

v

0xB6bC9e1D0b2fB96Ab7C47E04Cb0BE477410bC1f2

|

v

selector 0xb68d1809

|

v

ABI encoded URL

|

v

fingerprint-veri.info

The blockchain therefore functioned as a resolution/control layer, rather than merely being unrelated network noise.

10. Why Blockchain Resolution Matters

Traditional web malware often contains a hard-coded C2 or payload URL.

For example:

loader.js

|

+--> malware-domain.example

Blocking the destination can disrupt that specific infrastructure.

EtherHiding introduces another layer:

loader.js

|

v

blockchain contract

|

v

current backend

The compromised website does not necessarily need to be modified whenever the downstream backend changes.

Instead, the contract state can point the existing loader toward new infrastructure.

This has an important consequence for victimology.

Suppose a website is compromised at time T1.

At T2, the contract points to backend A.

At T3, the operator changes the contract to backend B.

At T4, a researcher discovers the website communicating with backend B.

The observation at T4 does not prove that the website was compromised at T4.

The infection may predate backend B considerably.

This became important when comparing apparently synchronized Italian victims.

11. BW Panel Retrieval Layer

After infrastructure resolution, the browser communicated with:

/api.php

using parameters such as:

?s=<hexadecimal value>

The primary active key was:

8402fb1338daab6a166e91aa8c92a20798acfa98fb75c5a2

Initial requests using this value returned large JavaScript modules.

During the captured session, approximately:

91 KB

and:

103 KB

were observed.

This behavior was substantially different from the smaller/no-op module associated with the other observed s= value.

The evidence suggested that s participated in module/configuration selection rather than functioning merely as a unique victim ID.

12. Internal Shared Key

Static analysis of the retrieved active module found the same value internally.

Conceptually:

K = "8402fb1338daab6a166e91aa8c92a20798acfa98fb75c5a2";

This was significant because the value therefore existed in at least two contexts:

NETWORK REQUEST

/api.php?s=K

and:

CLIENT MODULE

K = 8402...

The subsequent protocol analysis established a third context.

13. Callback Discovery

Later traffic contained a request with:

k=8402...

and an additional:

d=...

parameter.

This produced the relationship:

GET /api.php?s=K

|

v

JavaScript module

|

v

client execution

|

v

/api.php?k=K&d=...

The same K was therefore reused across protocol stages.

This became one of the core findings of the investigation.

14. Reverse Engineering the d Parameter

A complete request body of approximately 266 bytes was captured.

It contained:

d=eyJhY3Rpb24iOiJkbCIzImsiOiI4NDAY...

The value was Base64.

Decoding it produced:

{

"action": "dl",

```
"k": "8402fb1338daab6a166e91aa8c92a20798acfa98fb75c5a2",  
"os": "windows",  
"vid": "b9b28dd2-8d18-4322-9c0a-012210e465fd",  
"ref": "https://www.danielenittisotres.it/",  
"ray": "41065341e5e17c2b"  
}
```

This was one of the most useful pieces of protocol evidence collected during the research.

15. Callback Field Analysis

action

Observed:

"dl"

This clearly represents an operation identifier.

The precise semantic expansion of dl is not proven by the captured request alone.

It should therefore not be described definitively as “download” without additional evidence.

Status: CONFIRMED value / UNKNOWN exact semantics

k

Observed:

8402fb...

This is identical to the value previously used as:

s=8402fb...

and embedded inside the JavaScript client.

Status: CONFIRMED shared protocol key

os

Observed:

windows

This describes the operating-system environment.

Status: CONFIRMED

vid

Observed:

b9b28dd2-8d18-4322-9c0a-012210e465fd

The value follows UUID formatting.

Static client analysis also identified generation of a randomized UUID-like value.

The evidence therefore supports interpreting vid as a client/browser identifier.

Status: ASSESSED — HIGH CONFIDENCE

Observed:

<https://www.danielenittisotres.it/>

This directly identifies the victim-site context.

Status: CONFIRMED

ray

Observed:

41065341e5e17c2b

ray is transmitted independently from vid.

It therefore represents a distinct token/state field.

Its exact backend semantics cannot be determined from the client alone.

Status: CONFIRMED distinct field / exact semantics UNKNOWN

16. Multiple State Scopes

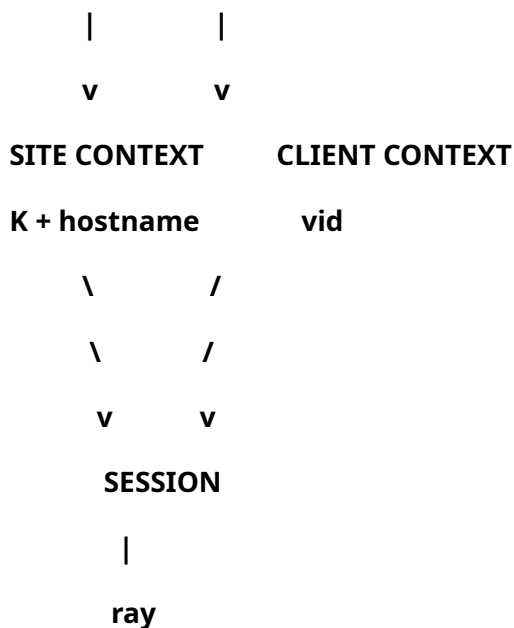
At this point the client no longer appeared to maintain a single identifier.

Instead, several different scopes were visible.

Conceptually:

FRAMEWORK / CONFIGURATION

|
K
|
+-----+



This distinction is important.

A framework can use one value to identify a configuration/module while simultaneously maintaining separate identifiers for:

- the compromised site;
- the browser/client;
- the current interaction/session.

The observed client was consistent with such a layered model.

17. Host-Derived State

Static analysis identified logic in which the shared key participated in state derivation together with:

`location.hostname`

The resulting state was deterministic for a given combination of key and hostname.

This suggests a site-scoped state value.

The same module separately created a randomized client identifier.

Consequently:

`K`

`+`

`hostname`

site-derived state

was distinct from:

random UUID

client identifier

and both were distinct from the transmitted:

ray

value.

This is an important finding but requires careful scoping.

The host-derived routine was statically confirmed in the analyzed module.

A second independent complete JavaScript module exposing the identical construction was not obtained.

Therefore:

The behavior is **CONFIRMED** in the primary sample, but **NOT PROVEN** as universal BW Panel behavior.

18. Discovery of the 17-Byte Response

One of the most interesting observations occurred after the active client had already communicated with the backend.

Earlier:

s=8402...

|

+--> ~91 KB JavaScript

|

+--> ~103 KB JavaScript

Later, during the same execution:

s=8402...

|

+--> 17 bytes

Initially, “17 bytes” alone was insufficient to determine what had happened.

Possible explanations included:

- **HTTP error;**
- **empty body;**
- **server message;**
- **truncated content;**
- **sandbox artifact;**
- **deliberate response.**

The response was therefore inspected directly.

19. Explicit JavaScript No-op

The complete 17-byte response was:

(function(){})();

This is valid JavaScript.

It defines and immediately invokes an empty function.

It intentionally performs no operation.

This eliminates several alternative explanations.

The response was not:

- **missing;**
- **malformed;**
- **an HTTP failure;**
- **a parsing artifact;**
- **a truncated active module.**

Instead, the backend deliberately returned executable JavaScript whose effect was nothing.

This is one of the strongest individual observations in the case study because it has a practical defensive implication.

A scanner observing only this later state could potentially interpret the endpoint as inactive even though the same key previously returned active code.

20. State-Dependent Module Delivery

This produces the following observed sequence:

GET ?s=K

|

+--> active module (~91 KB)

GET ?s=K

|

+--> active module (~103 KB)

client executes

|

v

callback / POST activity

k=K

d=<client context>

|

v

GET ?s=K

|

+--> (function(){})();

17-byte explicit no-op

The same key therefore did not correspond permanently to one static payload.

Something changed during the session.

The backend subsequently chose not to deliver the active module.

This allows the following conclusion:

Module delivery was state-dependent in the observed session.

Status: CONFIRMED

What cannot be concluded is *which* state caused the change.

21. What Triggered the No-op?

Several explanations are technically plausible.

The backend may have recorded that:

- the module had already been served;
- the client had completed a particular interaction;
- a session token had been consumed;
- the client was no longer eligible;
- replay suppression was activated;
- a browser fingerprint had already been processed.

However, the sandbox evidence does not expose the server-side decision logic.

Therefore all such explanations remain hypotheses.

The correct conclusion is:

State-dependent delivery: CONFIRMED

Exact gating condition: UNKNOWN

This distinction is central to the credibility of the research.

22. Reconstructed Protocol

The complete observed chain can now be represented as:

COMPROMISED WEBSITE

|

v

OBFUSCATED JAVASCRIPT

|

v

DEOBFUSCATION / DYNAMIC EXECUTION

|

v

POLYGON JSON-RPC

|

v

eth_call

|

v

0xB6bC9e1D0b2fB96Ab7C47E04Cb0BE477410bC1f2

|

v

selector 0xb68d1809

|

v

ABI-DECODED URL

|

v

ROTATING BACKEND

|

v

/api.php?s=K

|

v

ACTIVE BW PANEL CLIENT

|

```

+-----> site-derived state
|
+-----> randomized client ID
|
+-----> victim referrer
|
+-----> ray/session-like token
|
v
/api.php?k=K&d=<encoded context>
|
v
BACKEND STATE TRANSITION
|
v
/api.php?s=K
|
v
(function(){}());

```

This represents the central technical model produced by the investigation.

23. External Protocol Corroboration

A critical question was whether the protocol characteristics observed in the deep-dive sample were unique to that website. External public telemetry was reviewed during the wider research process, but this publication does not treat those external samples as primary evidence unless they are individually cited and classified.

Detailed cross-victim claims are intentionally excluded from this version until every external sample has a traceable source, a reproducible artifact, and an explicit confidence label.

Within the primary sample, K is reused across protocol stages while victim-site context is carried separately in `ref`. This is sufficient to reject a simplistic interpretation of K as the only victim identity field in this client, but it is not sufficient on its own to prove cross-victim reuse.

Framework-level or cross-victim reuse may be evaluated in a future cited appendix, but it is not promoted to a CONFIRMED finding in the present evidence matrix.

The primary evidence remains limited to the client/state relationships directly observed in the deep-dive sample.

Any future cross-victim correlation must be handled carefully.

Even if two sites share a key, contract or selector, that would not automatically prove that they were compromised:

- **at the same time;**
- **by the same initial-access technique;**
- **by the same human operator.**

At most, such overlap would support a delivery/framework relationship once the external evidence is documented.

24. What Is Original Here?

EtherHiding is not our discovery.

ClickFix is not our discovery.

BW Panel is not our discovery.

The Polygon selector is not claimed as our discovery.

Existing public research already describes major components of this ecosystem.

The contribution of this investigation is narrower.

Contribution 1 — Protocol-stage key reuse

The same shared K was traced from:

`?s=K`

to the JavaScript client and subsequently:

`k=K`

inside callback communication.

Contribution 2 — Callback structure

The encoded callback was decoded and mapped into:

action

k

os

vid

ray

showing distinct pieces of client and victim context.

Contribution 3 — Multiple state scopes

The analyzed module contains evidence consistent with separate:

- **shared/configuration state;**
- **host-derived state;**
- **randomized client identity;**
- **separate token/session state.**

Contribution 4 — Explicit state-dependent suppression

The same module key changed from delivering substantial active JavaScript to:

(function(){})();

within the same sandbox session.

The correct publication language is therefore:

under-described stateful behavior observed in an active BW Panel client

rather than:

“discovery of BW Panel.”

25. Victimology

Related victimology was considered during the wider research process, but this publication does not promote additional websites to confirmed-victim status unless their evidence is reproduced in this document. The victimology discussion therefore focuses on analytical methodology: correlation through a shared delivery framework is not equivalent to proof of a common compromise event.

The wider research considered whether correlated sites shared:

- **sector;**
- **CMS;**
- **hosting provider;**
- **geographic profile;**
- **plugin;**
- **vulnerability;**
- **compromise period.**

The wider research suggested heterogeneity, but that broader dataset is outside the primary evidence package of this paper.

WordPress was relevant to the wider investigation, but this document does not use unrepresented victim data to infer a universal compromise path.

Accordingly, no claim is made here that every related observation resulted from the same initial-access mechanism.

Any broader provider or sector clustering should be published separately with its own source-backed victim matrix.

This paper therefore does not advance the hypothesis:

one provider → one compromise → all victims

Nor does it use the current evidence package to claim that one WordPress vulnerability explains a broader cluster.

If a future public victim matrix is released, it should distinguish between evidence classes rather than labeling every correlated domain as infected:

DIRECTLY ANALYZED

PUBLICLY CONFIRMED COMPROMISE

STRONG INFRASTRUCTURE CORRELATION

and:

CROSS-SAMPLE PROTOCOL MATCH

rather than labeling every correlated domain as infected.

25.1 Deep-Dive Sample: Observed Domain and Infrastructure Matrix

This matrix is intentionally restricted to indicators explicitly present in the ANY.RUN report for danielennittisotres[.]it. `OBSERVED` means the indicator appeared in that sandbox telemetry; it does not by itself establish that every domain was independently compromised or exclusively attacker-controlled.

Indicator	Type	Role in primary evidence	Status	Confidence
danielennittisotres[.]it	Domain	Deep-dive sandbox sample / analyzed victim website	DIRECTLY ANALYZED	High
www.danielennittisotres[.]it	Domain	Victim-site traffic	OBSERVED	High
fingerprint-veri[.]info	Domain	Panel/backend traffic	OBSERVED	High
norediam[.]com	Domain	Sandbox domain/IOC list	OBSERVED	High
triapfog[.]com	Domain	Sandbox domain/IOC list	OBSERVED	High
mioiooo[.]icu	Domain	Traffic API observed	OBSERVED	High
clickzona[.]net	Domain	Sandbox domain/IOC list	OBSERVED	High
cloudflare-check[.]net	Domain	Sandbox domain/IOC list	OBSERVED	High
89.46.106.61	IP	Sandbox IP list	OBSERVED	High
58.64.137.69	IP	Sandbox IP list	OBSERVED	High
178.16.52.101	IP	Sandbox IP list	OBSERVED	High

Extended victimology rule: additional Italian or international domains should enter the public victim matrix only when each row has a traceable evidence source and an explicit status such as DIRECTLY ANALYZED, PUBLICLY CONFIRMED COMPROMISE, STRONG INFRASTRUCTURE CORRELATION, or CROSS-SAMPLE PROTOCOL MATCH.

26. The CVE Hypothesis

One research direction was to determine whether the victim cluster could reveal an actively exploited WordPress CVE.

This would have significantly increased the investigative value if supported.

The reasoning was straightforward:

Victim A

Victim B

Victim C

Victim D

|

v

common plugin/version

|

v

known vulnerability

However, the evidence did not converge strongly enough.

The available victim stacks did not establish one shared vulnerable component with sufficient confidence.

Consequently:

Single actively exploited CVE: NOT SUPPORTED

This is not a failed result.

Eliminating an attractive but unsupported hypothesis is part of good threat research.

27. Initial Access Versus Delivery Framework

One of the most important analytical distinctions produced by the research is between:

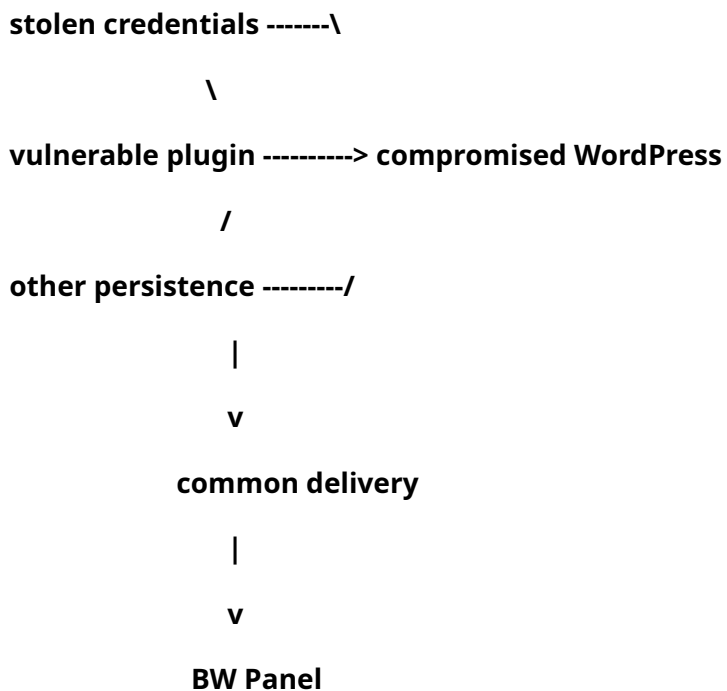
initial access

and:

post-compromise delivery.

The fact that several websites deliver the same BW Panel framework does not prove that the websites were compromised in the same way.

Conceptually:



Different initial-access methods can converge on the same malware-delivery infrastructure.

The present evidence is much stronger for the downstream framework than for the original compromise mechanism.

28. Attribution

Another tempting step would be to identify the threat actor behind the operation.

The evidence does not currently justify attribution to a named APT.

Infrastructure reuse can identify an ecosystem or framework.

It does not automatically identify the human operator.

A contract can be:

- shared;
- rented;
- resold;
- operated by an affiliate system;
- reused across campaigns.

Similarly, a traffic-distribution system can serve multiple downstream actors.

Therefore the report attributes the observed behavior at the framework/campaign ecosystem level, not to a specific APT.

Specific APT attribution: NOT SUPPORTED

This is intentionally conservative.

29. Detection Engineering

The investigation produced several useful defensive pivots.

29.1 Volatile indicators

These include domains such as:

fingerprint-veri[.]info

and individual backend IP addresses.

They remain useful for immediate investigation and blocking.

However, they are likely to change.

30. Durable Network Detection

A more robust behavioral chain is:

browser

|

v

Polygon JSON-RPC

|

v

eth_call

|

v

0xb68d1809

|

v

ABI-decoded URL

|

v

/api.php?s=<high entropy key>

|

v

same key later reused in

k=<key>&d=<encoded structure>

Each element individually can have benign explanations.

Their conjunction is much less common.

This makes correlation preferable to single-string detection.

31. Static JavaScript Detection

Potential static features include:

- numerical arrays;
- repeated XOR reconstruction;
- atob();
- String.fromCharCode;
- new Function;
- JSON-RPC structures;
- eth_call;
- multiple Polygon RPC URLs;
- selector 0xb68d1809.

Again, one feature should not be treated as sufficient.

For example:

atob()

is extremely common.

But:

atob

+

XOR

+

new Function

+

Polygon RPC

+

eth_call

+

known selector

is significantly more specific.

32. Detection Philosophy

The broader defensive conclusion is:

Hunt the protocol, not just the infrastructure.

An IOC such as:

fingerprint-veri.info

can disappear immediately.

The attacker can change an IP address.

The operator can change variable names.

XOR constants can be regenerated.

RPC endpoint ordering can be randomized.

But the operation still needs to perform its underlying tasks.

Those requirements create structural invariants.

Finding those invariants was therefore one of the main goals of this case study.

33. Primary IOC Set

The primary sandbox report identified the following domains among the observed traffic:

danielennittisotres[.]it

norediam[.]com

triapfog[.]com

fingerprint-veri[.]info

mioiooo[.]icu

clickzona[.]net

cloudflare-check[.]net

The report lists these domains directly.

The report additionally identified three IP addresses:

89.46.106.61

58.64.137.69

178.16.52.101

These should be treated as observed indicators, not necessarily as infrastructure exclusively controlled by one threat actor.

That distinction is important because sandbox network traces can include infrastructure contacted indirectly or through legitimate services.

34. MITRE ATT&CK Context

The sandbox explicitly records:

T1204.002

for manual execution by a user.

It additionally records discovery behavior associated with:

T1012 — Query Registry

and:

T1082 — System Information Discovery

including checks relating to the computer name and supported languages.

The ClickFix social-engineering mechanism is particularly relevant because the victim is induced to participate in the execution process.

This creates a behavioral sequence different from silent drive-by exploitation.

35. Limitations

Several limitations remain.

Server-side visibility

We do not possess the backend source code.

Therefore the exact gating logic responsible for the no-op cannot be proven.

Victim server visibility

We do not possess forensic images of the compromised WordPress servers.

Therefore exact initial access cannot be reconstructed.

Historical visibility

Sandbox observation dates do not necessarily correspond to compromise dates.

Public dataset bias

Public sandbox samples represent only activity submitted by researchers and automated systems.

Originality search

Failure to locate a publication describing an exact implementation detail does not prove nobody has previously observed it.

For this reason, the report uses:

“under-described”

rather than:

“never previously discovered.”

36. Evidence Matrix

Finding	Confidence
Primary ClickFix/EtherHiding behavior	CONFIRMED
Polygon eth_call	CONFIRMED
Target contract 0xB6bC...C1f2	CONFIRMED
0xb68d1809	CONFIRMED
Contract response → backend URL	CONFIRMED
s=6c720...	CONFIRMED
s=8402...	CONFIRMED
K inside active module	CONFIRMED
s=K → k=K reuse	CONFIRMED
Base64 d structure	CONFIRMED
action/k/os/vid/ref/ray	CONFIRMED
vid separate from ray	CONFIRMED
K + hostname state derivation	CONFIRMED in primary sample
randomized client ID	CONFIRMED in primary sample
~91/103 KB active modules	CONFIRMED
17-byte explicit no-op	CONFIRMED
state-dependent delivery	CONFIRMED in observed session
exact gating mechanism	UNKNOWN
host-derived routine framework-wide	NOT PROVEN
single common CVE	NOT SUPPORTED
exact initial access	UNKNOWN
named APT attribution	NOT SUPPORTED

37. Key Research Finding

The strongest defensible finding from this investigation can be summarized as follows:

This research documents under-described stateful behavior in an active BW Panel/ErrTraffic client. A shared key was observed across module retrieval and callback stages, while the client separately maintained victim-site, client and session context. During the same sandbox execution, requests using the same module key transitioned from substantial active JavaScript delivery to an explicit executable no-op, demonstrating state-dependent module delivery in the observed session.

38. Why the Finding Matters

The finding has implications beyond this particular campaign.

If malware-delivery infrastructure maintains state, repeated analysis of the same URL may produce different results.

For example:

Researcher / scanner

first interaction

|

v

active malicious module

while later:

same target

subsequent interaction

|

v

(function(){})(());

A scanner seeing only the second response might conclude that the endpoint is benign or inactive.

This can complicate:

- **automated crawling;**
- **sandbox reproducibility;**
- **IOC validation;**
- **malware collection;**

- incident-response reproduction;
- retrospective threat hunting.

The observation of an explicit no-op makes this particularly interesting because suppression occurs while maintaining a valid JavaScript response.

The server does not need to return an error.

It can simply return code that does nothing.

This creates the possibility of an analytical false negative when the state preceding the request is not preserved.

Editorial Evidence Boundary

Names of related websites from earlier investigations are intentionally not used as evidence-bearing samples in this paper unless their technical artifacts are reproduced here. This avoids importing confidence judgments or disclosure status from separate work into CS3.

39. Defensive Implications

Security teams investigating similar campaigns should preserve more than the final payload.

Useful artifacts include:

- complete HTTP request order;
- response sizes;
- query parameters;
- cookies;
- local storage;
- session storage;
- referrers;
- browser identifiers;
- blockchain RPC calls;
- smart-contract responses;
- decoded ABI data;
- timing between requests.

Running the same URL repeatedly without preserving client state may not reproduce the original behavior.

Similarly, a threat-intelligence scanner should not automatically mark an endpoint clean because a previously malicious key now returns:

```
(function({})){}
```

The response may indicate suppression, not remediation.

This distinction could be particularly relevant for automated systems that perform repeated retrieval from the same analysis environment.

39.1 Responsible Disclosure and Victim Handling

The website documented in the primary sandbox analysis is treated as potential victim/compromised web infrastructure rather than attacker-controlled infrastructure unless separate evidence demonstrates otherwise. Disclosure status is intentionally kept distinct from technical classification. This public report does not infer successful notification or remediation from changes in observed malicious behavior; any disclosure status should be documented separately from the technical findings using only verifiable communication records.

40. Future Research

Several unanswered questions remain valuable for future work.

Q1

What exact server-side condition transforms active delivery into the 17-byte no-op?

Q2

What are the precise semantics of ray?

Q3

Is the host-derived state construction present across other current BW Panel modules?

Q4

How long does server-side state persist?

Q5

Does changing the client-generated identifier alter module delivery?

Q6

Does changing the victim hostname while preserving K alter backend behavior?

Q7

Can individual BW Panel deployments be clustered reliably by protocol implementation rather than infrastructure?

Q8

Can server-side forensic evidence from a future consenting victim identify the actual initial-access mechanism?

These questions are intentionally left open.

They represent potential follow-up research rather than missing conclusions.

41. Conclusion

This investigation began with compromised websites and apparently conventional ClickFix behavior.

It ultimately revealed a considerably more structured delivery architecture.

The analyzed chain combined:

- **compromised web infrastructure;**
- **polymorphic JavaScript;**
- **blockchain-based infrastructure resolution;**
- **Polygon smart-contract interaction;**
- **rotating backend infrastructure;**
- **modular JavaScript delivery;**
- **client-side state;**
- **encoded callback communication;**
- **state-dependent module suppression.**

The most valuable result was not identification of another malicious domain.

It was understanding how the pieces interact.

The observed client reused a shared key across multiple protocol stages while separately transmitting victim, client and token/session context.

The backend subsequently changed the content returned for that same key from large active JavaScript modules to an explicit no-op.

This behavior provides evidence that the delivery framework maintains or reacts to state during the infection process.

At the same time, several attractive conclusions could not be supported.

The research did not establish:

- **one common exploited CVE;**
- **the exact compromise mechanism for every victim;**
- **a specific named APT;**
- **the precise backend gating algorithm.**

Those limitations are preserved deliberately.

Threat research becomes more useful when UNKNOWN remains UNKNOWN.

The resulting case study therefore represents not simply IOC collection, but a reconstruction of part of the delivery protocol and its state model.

41.1 Evidence and Artifact Index

ID	Artifact	Evidence class	Use in paper
E-01	ANY.RUN primary report	Primary sandbox evidence	Classification, URLs, domains, IPs, ATT&CK
E-02	Captured `/api.php?s=...` traffic	Primary network evidence	Module retrieval keys and response behavior
E-03	Captured `k=<K>&d=...` request	Primary network evidence	Cross-stage key reuse and encoded client context
E-04	Decoded Base64 `d` object	Derived from primary capture	action, k, os, vid, ref, ray
E-05	Obfuscated JavaScript samples	Primary static evidence	XOR/Base64/dynamic execution and client logic
E-06	17-byte JavaScript response	Primary network evidence	Explicit no-op `(function({})){}`
E-07	Polygon JSON-RPC / `eth_call` capture	Primary network evidence	Contract/selector resolution path

41.2 References and Source Hygiene

Primary evidence for this paper consists of the sandbox report and artifacts captured during the investigation. Third-party research is used only for ecosystem context and does not upgrade a primary finding beyond what the captured evidence supports.

Core reference classes used for contextual interpretation include the primary ANY.RUN analysis and MITRE ATT&CK technique definitions. Any external ecosystem research cited in future revisions should be accompanied by a complete, verifiable bibliographic entry.

42. Final Research Status

Case Study: CS3

Research status: FROZEN

Technical investigation: COMPLETE

Primary finding: SUPPORTED

Cross-sample corroboration: PRESENT

Initial-access attribution: UNRESOLVED

APT attribution: NOT SUPPORTED

Publication readiness: HIGH

Final classification

Portfolio-grade independent threat research with an original, conservatively scoped protocol-level finding.

Suggested Citation

Salvadori, Andrea (BresshSec).

Tracking an Active ClickFix / EtherHiding Operation: Reverse Engineering BW Panel Stateful Delivery Across Compromised Italian Web Infrastructure.

Independent Threat Research, September 2026.

Release Notes

This release preserves the confidence model and distinguishes directly analyzed evidence, observed indicators, infrastructure correlation and future cross-sample work. Additional victim sites are not presented as evidence-bearing samples unless their artifacts and citations are included.

Revision v1.5 - Public Release

This public release centers the evidence base on the documented deep-dive sandbox sample, excludes unsupported cross-sample claims, preserves the evidence-limited EtherHiding reconstruction, and keeps unknown server-side gating, initial access and named-actor attribution explicitly unresolved.